In [21]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error
import numpy as np
import subprocess
import sys
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

I am using pandas to bring in data from the csv into a dataframe. Once done, I will quickly preview the first few rows to confirm it loaded correctly.

In [ ]:
```
df = pd.read_csv("GenAI/us_retail_sales.csv")
df.head()
```

Out[ ]:

	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DE
0	1992	146925	147223	146805	148032	149010	149800	150761.0	151067.0	152588.0	153521.0	153583.0	155614.
1	1993	157555	156266	154752	158979	160605	160127	162816.0	162506.0	163258.0	164685.0	166594.0	168161.
2	1994	167518	169649	172766	173106	172329	174241	174781.0	177295.0	178787.0	180561.0	180703.0	181524.
3	1995	182413	179488	181013	181686	183536	186081	185431.0	186806.0	187366.0	186565.0	189055.0	190774.
4	1996	189135	192266	194029	194744	196205	196136	196187.0	196218.0	198859.0	200509.0	200174.0	201284.

Now I am checking the basic shape and column names so I know what I am working with.

In [2]:
```
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

```
Shape: (30, 13)
Columns: ['YEAR', 'JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN', 'JUL', 'AUG', 'SEP', 'OCT', 'NOV', 'DEC']
```

I am checking anomalies like missing values, duplicate rows, and data types to make sure the data is free from issues before analysis.

In [3]:
```
print("Missing values by column:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nData types:")
print(df.dtypes)
```

```
Missing values by column:
YEAR    0
JAN     0
FEB     0
MAR     0
APR     0
MAY     0
JUN     0
JUL     1
AUG     1
SEP     1
OCT     1
NOV     1
DEC     1
dtype: int64

Duplicate rows: 0

Data types:
YEAR      int64
JAN       int64
FEB       int64
MAR       int64
APR       int64
MAY       int64
JUN       int64
JUL     float64
AUG     float64
SEP     float64
OCT     float64
NOV     float64
DEC     float64
dtype: object
```

The data is in wide format, so I am converting it to long format to make time series analysis easier.

In [4]:
```
month_order = ["JAN","FEB","MAR","APR","MAY","JUN","JUL","AUG","SEP","OCT","NOV","DEC"]

df_long = df.melt(
    id_vars="YEAR",
    value_vars=month_order,
    var_name="MONTH",
    value_name="SALES"
)

month_map = {m: i+1 for i, m in enumerate(month_order)}
df_long["MONTH_NUM"] = df_long["MONTH"].map(month_map)
df_long["DATE"] = pd.to_datetime(dict(year=df_long["YEAR"], month=df_long["MONTH_NUM"], day=1))

df_long = df_long.sort_values("DATE").dropna(subset=["SALES"]).reset_index(drop=True)
df_long.head()
```

Out[4]:

	YEAR	MONTH	SALES	MONTH_NUM	DATE
0	1992	JAN	146925.0	1	1992-01-01
1	1992	FEB	147223.0	2	1992-02-01
2	1992	MAR	146805.0	3	1992-03-01
3	1992	APR	148032.0	4	1992-04-01
4	1992	MAY	149010.0	5	1992-05-01

I am doing a quick EDA by looking at summary statistics. This helps me understand spread, center, and possible outliers.

In [5]:
```
df_long["SALES"].describe()
```

Out[5]:
```
count       354.000000
mean     307006.573446
std       94335.828235
min      146805.000000
25%      231402.000000
50%      309534.500000
75%      378193.750000
max      562269.000000
Name: SALES, dtype: float64
```

I am filtering the data to start from July 1992 to ensure complete years. This removes the 6 missing values and keeps data balanced.

In [6]:
```python
# Remove incomplete data (2021 JUL-DEC are missing)
df_clean = df_long.dropna(subset=["SALES"]).copy()

# Convert SALES column to int64
df_clean["SALES"] = df_clean["SALES"].astype("int64")

# Verify the conversion
print("Data types after conversion:")
print(df_clean.dtypes)

print("\nSample data:")
print(df_clean.head())

print("Shape before:", df_long.shape)
print("Shape after dropna:", df_clean.shape)

# Verify no more missing values
print("\nMissing values:", df_clean["SALES"].isnull().sum())

# just keep 29 full years of data (199207-202106)
df_clean = df_clean[df_clean["DATE"] >= "1992-07-01"].copy()
print("Shape after filtering date:", df_clean.shape)

# Check date range
print("Date range:", df_clean["DATE"].min(), "to", df_clean["DATE"].max())
```

```
Data types after conversion:
YEAR                    int64
MONTH                  object
SALES                   int64
MONTH_NUM               int64
DATE           datetime64[ns]
dtype: object

Sample data:
   YEAR MONTH   SALES  MONTH_NUM       DATE
0  1992   JAN  146925          1 1992-01-01
1  1992   FEB  147223          2 1992-02-01
2  1992   MAR  146805          3 1992-03-01
3  1992   APR  148032          4 1992-04-01
4  1992   MAY  149010          5 1992-05-01
Shape before: (354, 5)
Shape after dropna: (354, 5)

Missing values: 0
Shape after filtering date: (348, 5)
Date range: 1992-07-01 00:00:00 to 2021-06-01 00:00:00
```

Now I am confirming the cleaned data has balanced complete years.

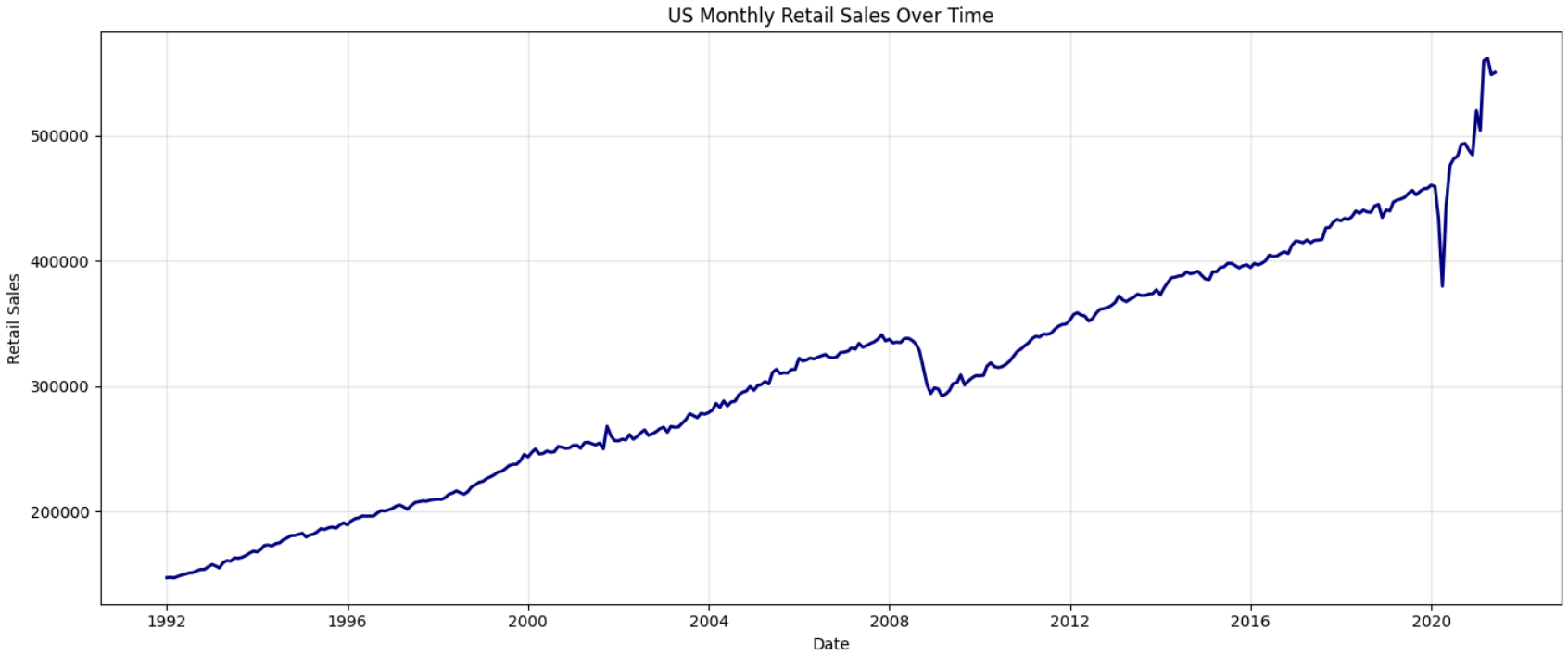
In [7]:
```python
# Verify we have complete years
years_list = df_clean["YEAR"].unique()
print(f"Years covered: {years_list[0]} to {years_list[-1]} ({len(years_list)} years)")
print(f"Total months: {len(df_clean)} (should be 29 years × 12 = 348)")

# Check if all years have 12 months
months_per_year = df_clean.groupby("YEAR").size()
print("\nMonths per year:")
print(months_per_year)
```

```
Years covered: 1992 to 2021 (30 years)
Total months: 348 (should be 29 years × 12 = 348)

Months per year:
YEAR
1992     6
1993    12
1994    12
1995    12
1996    12
1997    12
1998    12
1999    12
2000    12
2001    12
2002    12
2003    12
2004    12
2005    12
2006    12
2007    12
2008    12
2009    12
2010    12
2011    12
2012    12
2013    12
2014    12
2015    12
2016    12
2017    12
2018    12
2019    12
2020    12
2021     6
dtype: int64
```

I am plotting the full retail sales trend over time. This helps me observe trend, seasonality, and unusual changes.

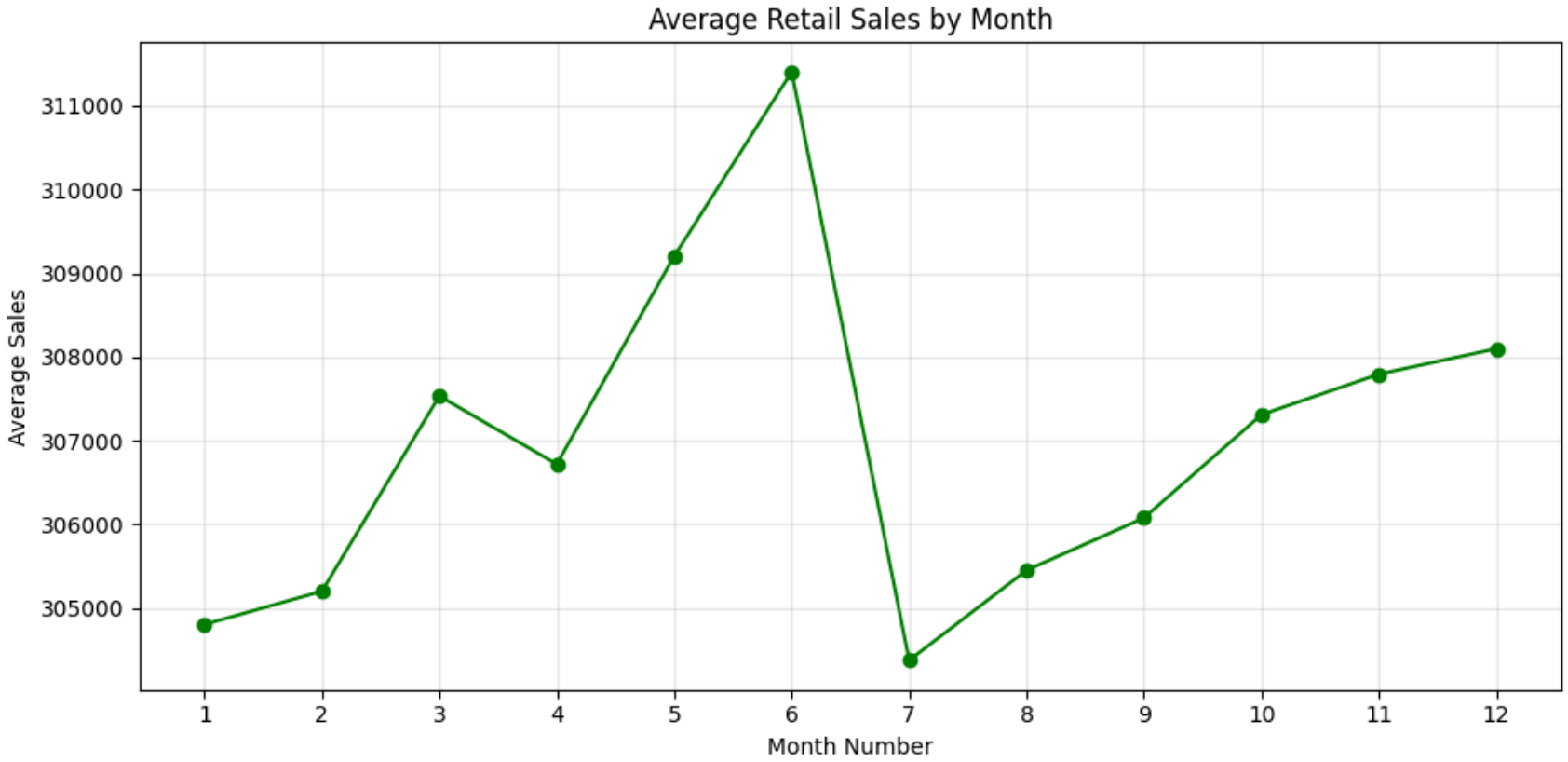
In [8]:
```
plt.figure(figsize=(14,6))
plt.plot(df_long["DATE"], df_long["SALES"], color="navy", linewidth=2)
plt.title("US Monthly Retail Sales Over Time")
```

```
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



I am doing one more quick EDA check by looking at average sales by month. This helps confirm seasonal behavior.

In [9]:
```
monthly_avg = df_long.groupby("MONTH_NUM")["SALES"].mean().reindex(range(1,13))

plt.figure(figsize=(10,5))
plt.plot(monthly_avg.index, monthly_avg.values, marker="o", color="green")
plt.title("Average Retail Sales by Month")
plt.xlabel("Month Number")
plt.ylabel("Average Sales")
plt.xticks(range(1,13))
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



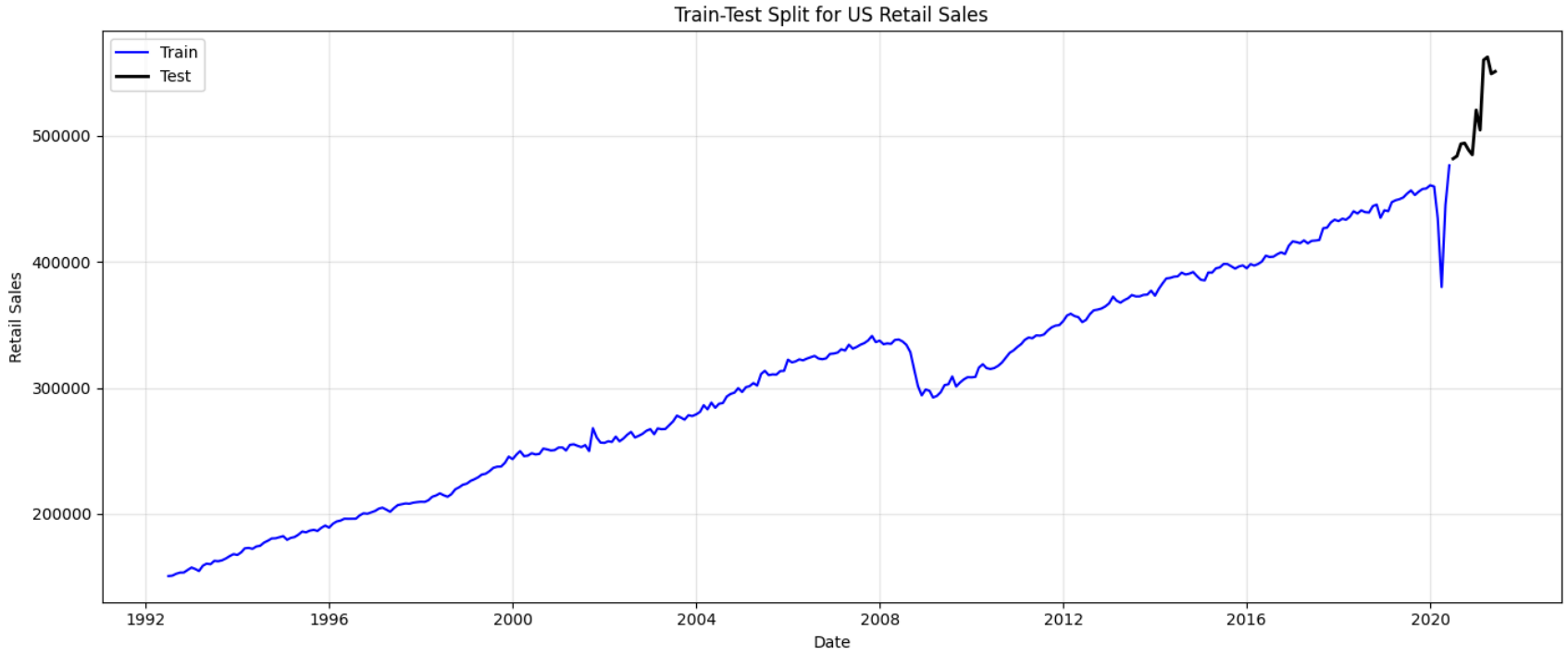
Now I am splitting the data into train and test sets. I am using July 2020 to June 2021 as test data exactly as required.

In [17]:
```
# Keep only the columns needed for modeling
ts = df_clean[["DATE", "SALES"]].copy().set_index("DATE")

# Train: up to June 2020
train = ts.loc[: "2020-06-01", "SALES"].copy()

# Test: July 2020 to June 2021
test = ts.loc["2020-07-01":"2021-06-01", "SALES"].copy()

print("Train period:", train.index.min().date(), "to", train.index.max().date(), "| rows:", len(train))
print("Test period:", test.index.min().date(), "to", test.index.max().date(), "| rows:", len(test))
```

```
Train period: 1992-07-01 to 2020-06-01 | rows: 336
Test period: 2020-07-01 to 2021-06-01 | rows: 12
```

I am plotting train and test together to visually verify the split.

In [18]:
```
plt.figure(figsize=(14,6))
plt.plot(train.index, train, label="Train", color="blue")
plt.plot(test.index, test, label="Test", color="black", linewidth=2)
plt.title("Train-Test Split for US Retail Sales")
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Here I am building a simple seasonal naive forecasting model. This model predicts each month using the value from the same month last year and acts as a baseline.

In [22]:
```
# Seasonal naive forecast for test period:
# Jul 2020–Jun 2021 predicted by Jul 2019–Jun 2020 actuals
```

```
pred = ts.loc["2019-07-01":"2020-06-01", "SALES"].copy()
pred.index = test.index
pred.name = "PRED"

pred.head()
```

Out[22]:
```
DATE
2020-07-01    454012
2020-08-01    456500
2020-09-01    452849
2020-10-01    455486
2020-11-01    457658
Name: PRED, dtype: int64
```

Now I am calculating RMSE to evaluate how close predictions are to actual test values.

In [ ]:
```
rmse = np.sqrt(mean_squared_error(test, pred))
print(f"RMSE: {rmse:,.2f}")
```

```
RMSE: 80,273.26
```

I am comparing actual vs predicted values month by month for the test period.

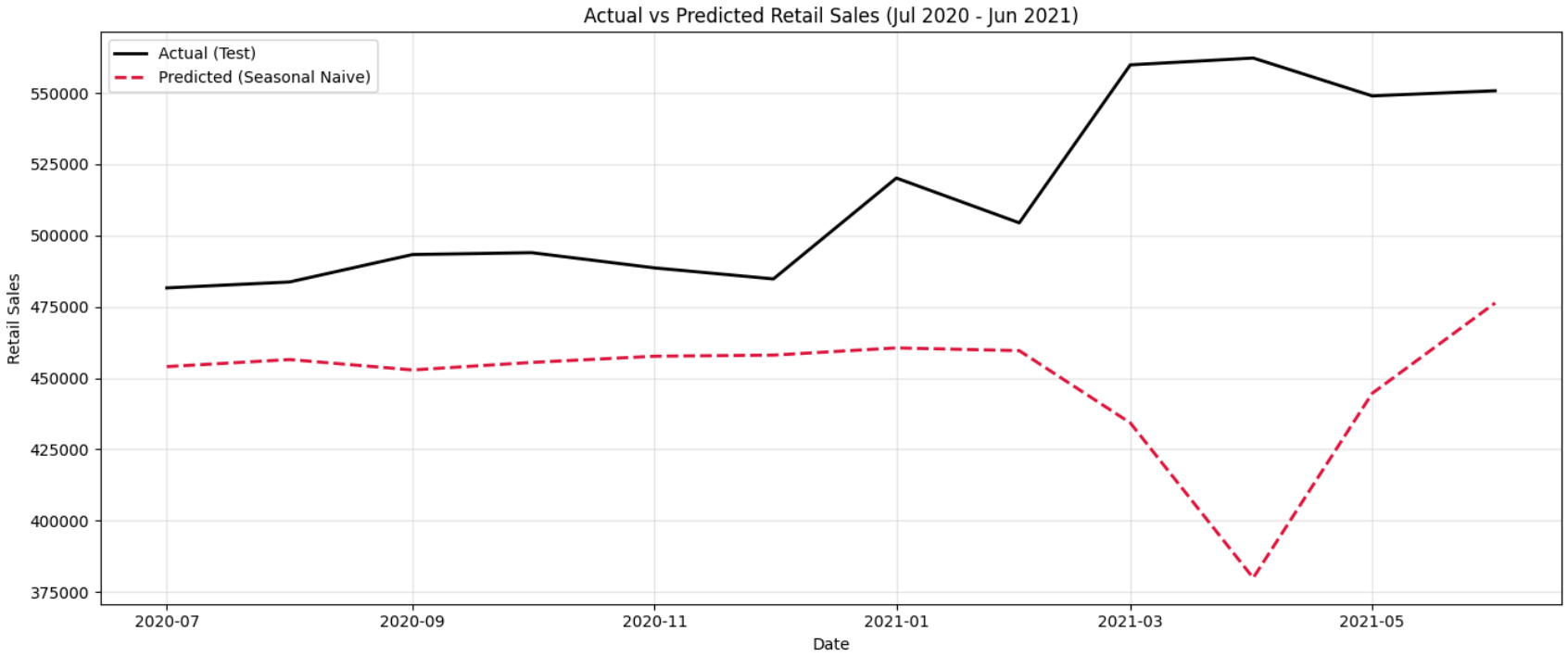
In [14]:
```
comparison = pd.DataFrame({
    "Actual": test,
    "Predicted": pred,
    "Error": test - pred
})
comparison
```

Out[14]:

DATE	Actual	Predicted	Error
2020-07-01	481627.0	454012.0	27615.0
2020-08-01	483716.0	456500.0	27216.0
2020-09-01	493327.0	452849.0	40478.0
2020-10-01	493991.0	455486.0	38505.0
2020-11-01	488652.0	457658.0	30994.0
2020-12-01	484782.0	458055.0	26727.0
2021-01-01	520162.0	460586.0	59576.0
2021-02-01	504458.0	459610.0	44848.0
2021-03-01	559871.0	434281.0	125590.0
2021-04-01	562269.0	379892.0	182377.0
2021-05-01	548987.0	444631.0	104356.0
2021-06-01	550782.0	476343.0	74439.0

I am plotting actual and predicted values for the test period. This helps me visually check model performance.

In [15]:
```
plt.figure(figsize=(14,6))
plt.plot(test.index, test, label="Actual (Test)", color="black", linewidth=2)
plt.plot(pred.index, pred, label="Predicted (Seasonal Naive)", color="crimson", linestyle="--", linewidth=
plt.title("Actual vs Predicted Retail Sales (Jul 2020 - Jun 2021)")
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Final observations:

1. The model captures yearly seasonality because it reuses same-month-last-year values.
2. It misses sudden trend shifts after the 2020 shock, so errors are larger in rebound months.
3. RMSE gives the final summary of prediction accuracy on the required test window.

I am now building a SARIMA model as an alternative to the simple seasonal naive approach. SARIMA captures trend and seasonality more flexibly. I am fitting a SARIMA model on the training data. SARIMA(p,d,q)(P,D,Q,s) where s=12 for monthly seasonality.

In [29]:
```
# Fit SARIMA model on training data
# (1,1,1)(1,1,1,12) is a common starting point for monthly data
model = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
sarima_fit = model.fit(disp=False)

print("SARIMA Model Summary:")
print(sarima_fit.summary())
```

```
/Users/pyadav/Library/Python/3.12/lib/python/site-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarn
ing: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/Users/pyadav/Library/Python/3.12/lib/python/site-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarn
ing: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
SARIMA Model Summary:
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                               SALES   No. Observations:                  336
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood               -3298.968
Date:                            Sun, 10 May 2026   AIC                           6607.935
Time:                                    17:13:59   BIC                           6626.824
Sample:                                07-01-1992   HQIC                          6615.475
                                     - 06-01-2020
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.6665      0.178      3.744      0.000       0.318       1.015
ma.L1         -0.7403      0.175     -4.232      0.000      -1.083      -0.397
ar.S.L12       0.5175      0.072      7.236      0.000       0.377       0.658
ma.S.L12      -0.8576      0.118     -7.238      0.000      -1.090      -0.625
sigma2      4.646e+07   2.59e-08   1.79e+15      0.000    4.65e+07    4.65e+07
===================================================================================
Ljung-Box (L1) (Q):                   0.15   Jarque-Bera (JB):             20821.04
Prob(Q):                              0.70   Prob(JB):                         0.00
Heteroskedasticity (H):               5.99   Skew:                             0.11
Prob(H) (two-sided):                  0.00   Kurtosis:                        42.33
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 5.84e+29. Standard errors may be
unstable.
```

The above warning is because Outer Product of Gradients struggled to adjust for 2020 extreme outliers hence even sigma2 as well kid of unstable above. Lets see if it converges on oim type.

In [30]:
```
# Fit SARIMA model with conv_type='oim' to get standard errors
```

```python
model = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
sarima_fit_oim = model.fit(disp=False, cov_type='oim')

print("SARIMA Model Summary:")
print(sarima_fit_oim.summary())
```

```
/Users/pyadav/Library/Python/3.12/lib/python/site-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarn
ing: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/Users/pyadav/Library/Python/3.12/lib/python/site-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarn
ing: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
SARIMA Model Summary:
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                SALES   No. Observations:                  336
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood               -3298.968
Date:                            Sun, 10 May 2026   AIC                           6607.935
Time:                                    17:14:06   BIC                           6626.824
Sample:                                07-01-1992   HQIC                          6615.475
                                     - 06-01-2020
Covariance Type:                              oim
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.6665      0.218      3.061      0.002       0.240       1.093
ma.L1         -0.7403      0.196     -3.772      0.000      -1.125      -0.356
ar.S.L12       0.5175      0.100      5.174      0.000       0.321       0.714
ma.S.L12      -0.8576      0.097     -8.828      0.000      -1.048      -0.667
sigma2      4.646e+07   5.39e-10   8.62e+16      0.000    4.65e+07    4.65e+07
===================================================================================
Ljung-Box (L1) (Q):                   0.15   Jarque-Bera (JB):             20821.04
Prob(Q):                              0.70   Prob(JB):                         0.00
Heteroskedasticity (H):               5.99   Skew:                             0.11
Prob(H) (two-sided):                  0.00   Kurtosis:                        42.33
===================================================================================

Warnings:
[1] Covariance matrix calculated using the observed information matrix (complex-step) described in Harvey
(1989).
[2] Covariance matrix is singular or near-singular, with condition number 5.01e+33. Standard errors may be
unstable.
```

Oim made the condition number even worse, so as of now lets stay with old results

I am using the fitted SARIMA model to make predictions on the test period.

In [31]:
```
# Forecast on test period
sarima_pred = sarima_fit.get_forecast(steps=len(test)).predicted_mean

print("SARIMA Predictions (first 5 months):")
print(sarima_pred.head())
```

```
SARIMA Predictions (first 5 months):
2020-07-01    476334.293513
2020-08-01    476484.130134
2020-09-01    473281.755670
2020-10-01    474660.328524
2020-11-01    475921.435266
Freq: MS, Name: predicted_mean, dtype: float64
```

I am calculating RMSE for the SARIMA model to compare with Seasonal Naive.

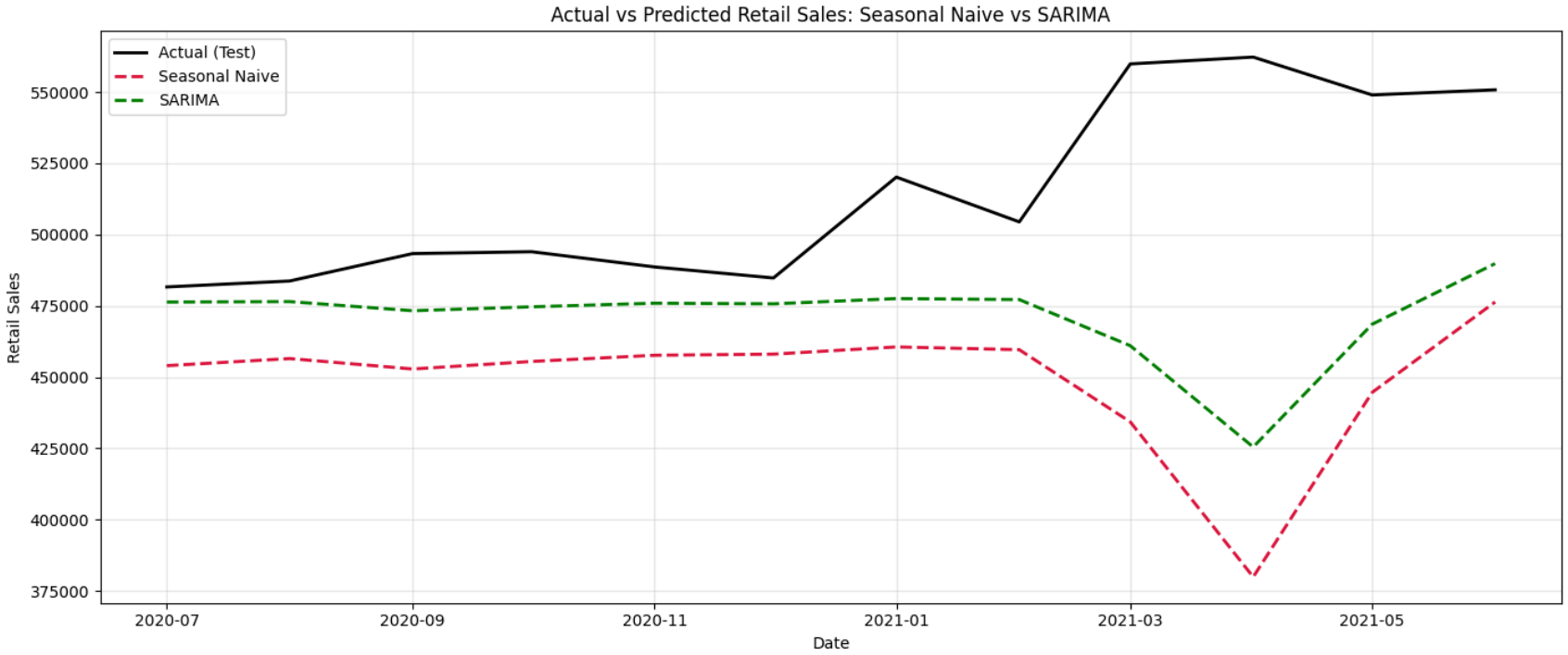
In [32]:
```
sarima_rmse = np.sqrt(mean_squared_error(test, sarima_pred))
print(f"SARIMA RMSE: {sarima_rmse:,.2f}")

# Compare both models
print("\n--- Model Comparison ---")
print(f"Seasonal Naive RMSE: {rmse:,.2f}")
print(f"SARIMA RMSE:        {sarima_rmse:,.2f}")
print(f"Better model:       {'SARIMA' if sarima_rmse < rmse else 'Seasonal Naive'}")
```

```
SARIMA RMSE: 59,394.13

--- Model Comparison ---
Seasonal Naive RMSE: 80,273.26
SARIMA RMSE:        59,394.13
Better model:       SARIMA
```

I am plotting both models' predictions side by side to visualize the comparison.

In [33]:
```
plt.figure(figsize=(14,6))
plt.plot(test.index, test, label="Actual (Test)", color="black", linewidth=2)
plt.plot(pred.index, pred, label="Seasonal Naive", color="crimson", linestyle="--", linewidth=2)
```

```
plt.plot(sarima_pred.index, sarima_pred, label="SARIMA", color="green", linestyle="--", linewidth=2)
plt.title("Actual vs Predicted Retail Sales: Seasonal Naive vs SARIMA")
plt.xlabel("Date")
plt.ylabel("Retail Sales")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Even with condition number bad, SARIMA performed better than Seasonal Naive because it adapts to trends, specially 2020 covid numbers.